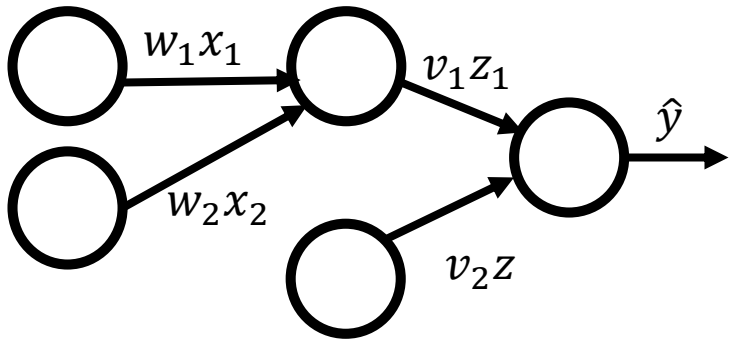
Two-layer backpropagation

- Backprop = gradient descent + chain rule
  - Solution and algorithm for two layers is not too complex (see Daume)
- Objective:

$$\min_{\mathbf{W},v} \sum_n \frac{1}{2}\left(y_n - \underbrace{\sum_i v_i f(\boldsymbol{w}_i \cdot \boldsymbol{x}_n)}_{\text{This is the prediction } \hat{y}_n}\right)^2$$

  - n indexes observations
  - i indexes hidden units
  - v is second layer weights
  - f is the sigmoid function
  - $\boldsymbol{w}_i$ is the vector of weights feeding into node i
  - $\mathbf{W}$ is first layer weights

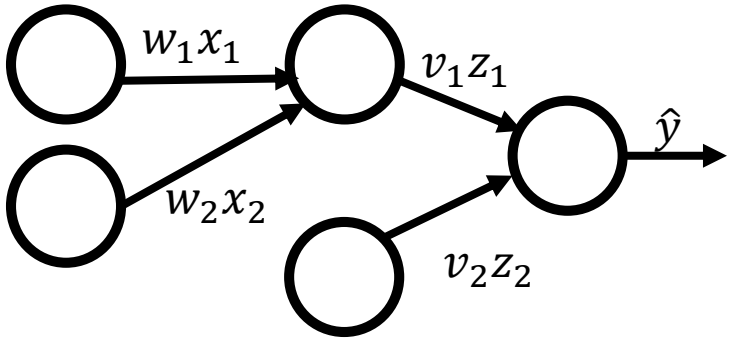
$$z_1 = f(w_1x_1 + w_2x_2)$$
$$z_2 = f(w_3x_1 + w_4x_2)$$



Two-layer backpropagation

- Objective:

$$\min_{\mathbf{W},v} \sum_n \frac{1}{2}\left(y_n - \sum_i v_i f(\boldsymbol{w}_i \cdot \boldsymbol{x}_n)\right)^2$$



- Loss:

$$\mathcal{L}(\mathbf{W}) = \frac{1}{2}\left(y - \sum_i v_i f(\boldsymbol{w}_i \cdot \boldsymbol{x})\right)^2$$

- Apply chain rule wrt. weights w:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}_i} = \frac{\partial \mathcal{L}}{\partial f_i}\frac{\partial f_i}{\partial \boldsymbol{w}_i}$$

$$\frac{\partial \mathcal{L}}{\partial f_i} = -\left(y - \sum_i v_i f(\boldsymbol{w}_i \cdot \boldsymbol{x})\right) v_i = -ev_i$$

$$\frac{\partial f_i}{\partial \boldsymbol{w}_i} = f'(\boldsymbol{w}_i \cdot \boldsymbol{x})\boldsymbol{x}$$

prediction error $(y - \hat{y})$

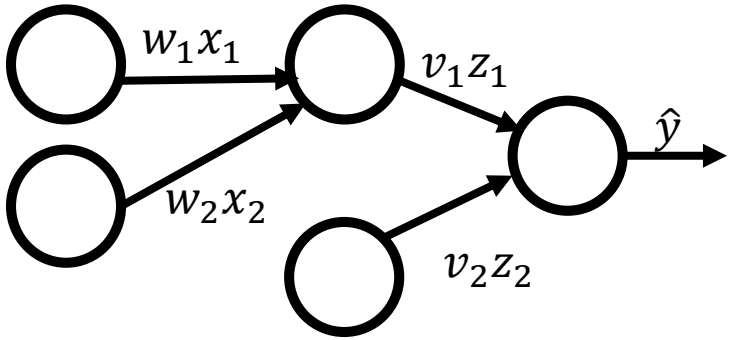
- In other words:

$$\nabla_{\boldsymbol{w}_i} = -ev_i f'(\boldsymbol{w}_i \cdot \boldsymbol{x})\boldsymbol{x}$$

Learning multilayer weights

- Solution with two layers

$$\nabla_{w_i} = -ev_i f'(\boldsymbol{w}_i \cdot \boldsymbol{x})\boldsymbol{x}$$



- Does this make sense?
  - If predictive error (e) is small, take small steps
  - If v_i is small, hidden unit i has little influence on output, gradient should be small
  - If e or v_i changes sign, gradient should also flip sign

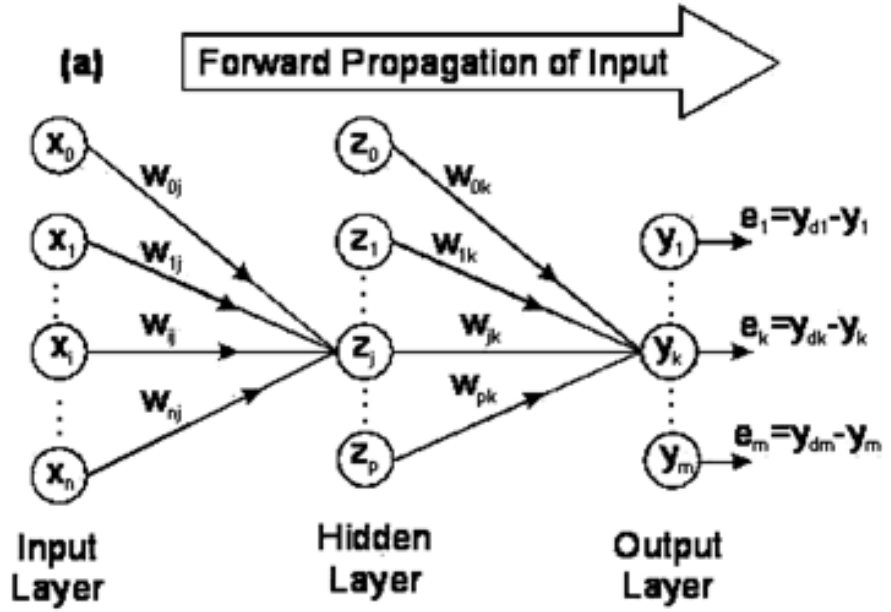
Backpropagation

- How to generalize from two layers?
- Sketch of procedure

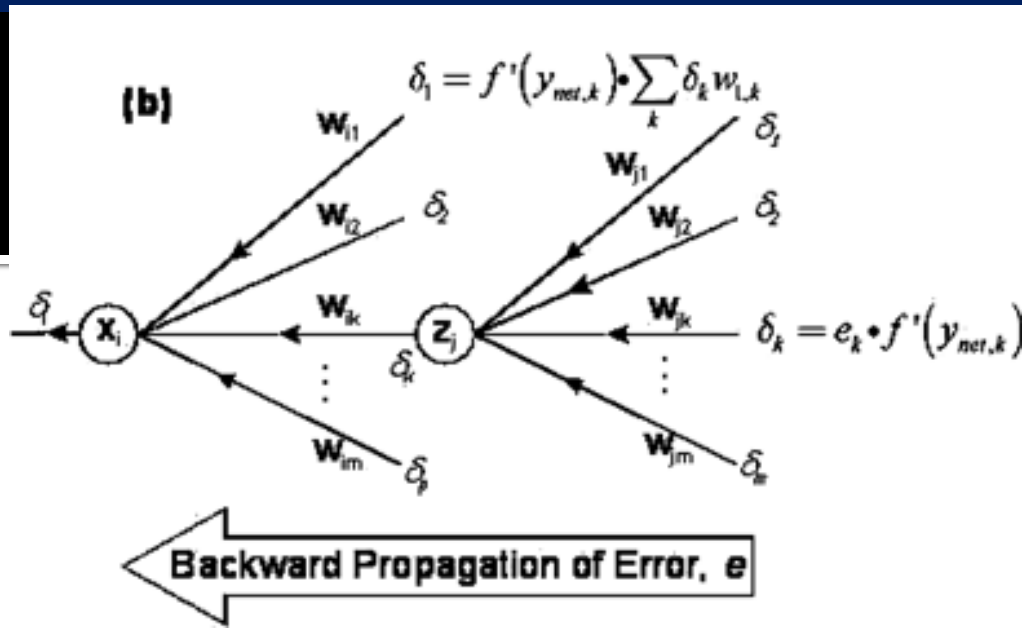
1. Forward Propagation -> Outputs
2. Backward Propagation -> Generate "deltas"
3. Weight Update -> same as in gradient descent

Intuition

- **Forward Propagation**
  - Given a training example (X_1 , …, X_n) and output Y_i:
  - Propagate inputs/activations forward, applying sigmoid function on dot products, calculate residuals



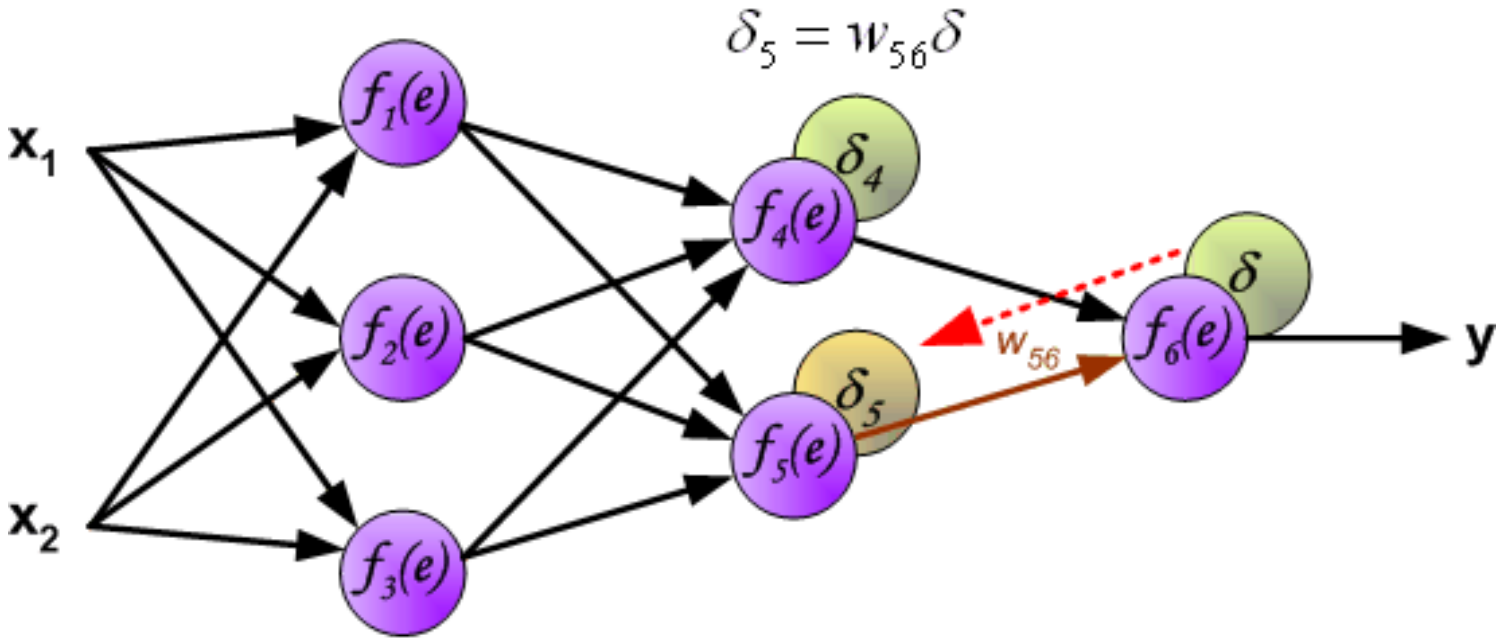
Intuition



- **Backward Propagation**
  - For a single training example i:
    - $\text{Cost}(i) = Y \cdot \log \hat{Y}_i + (1 - Y_i) \log(1 - \hat{Y}_i)$
    - i.e., how close is output to actual value?
  - Idea is to propagate costs backwards to earlier nodes
    - Compute δ_{jK} = "error" of j^{th} node in K^{th} (output) layer
      - $\delta_{jK} = Y_{jK}(1 - Y_{jK})(\hat{Y}_{jK} - Y_{jK})$
    - Note: deriving these partials is a bit complex, but mostly just chain rule + gradient descent (see ESL ch. 11)

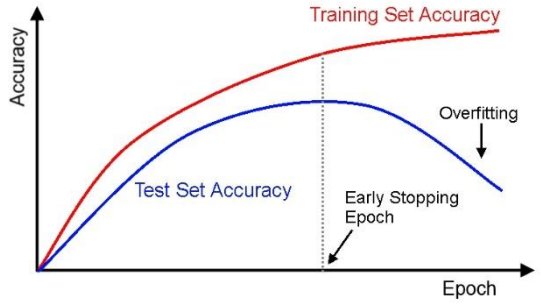
Intuition

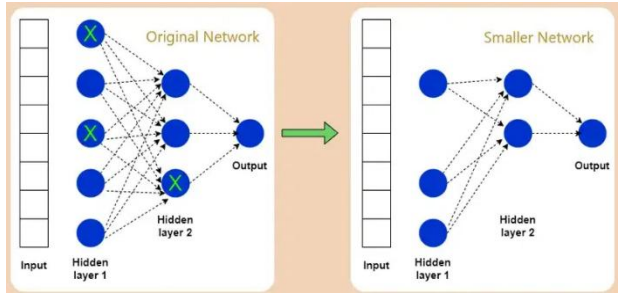
- **Update weights**
  - For each hidden unit j in k^{th} layer, update each weight as $+\eta\delta_{jk}x_i$



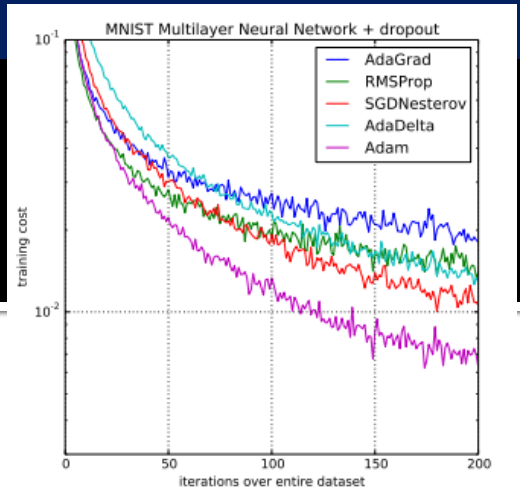
Neural Networks: Issues

- Non-convex, sensitive to initialization
  - Common solutions: Randomize initialization (small random/uniform weights), train multiple networks
- Avoiding overfitting
  - Early stopping
  - Include fewer layers, weights per layer
  - Penalize large weights (e.g., L1/L2 regularization)
  - **Dropout** – randomly drop neurons during training





Neural Networks: Issues



- Vanishing & Exploding **Gradients**
  - Problem: Gradients get too small (vanish) or too large (explode) in backprop
  - Solution: Activation functions like ReLU, Batch Normalization
- Choosing the right **Learning Rate**
  - If learning rate too high, network doesn't converge; if too low, training is slow
  - Solution: Using learning rate schedules, adaptive optimizers (Adam, RMSprop)
- **Computational Cost**
  - Problem: Large networks require significant computational power
  - Solutions: GPU/TPU computing, model parallelization, gradient checkpointing, stochastic gradient descent (SGD), mini-batch training

Tuning networks: Not easy to get it right!

- Several hyperparameters to consider:
  - Learning rate – can tune, or use adaptive optimizers
  - How to initialize – randomize weights, train several networks
  - Regularization techniques – weight decay, dropout
  - Batch size – small batches can be noisy, large batches can be slow
  - How many layers – generalization vs. overfitting
  - How many units per layer – generalization vs. overfitting
  - When to stop?

Tuning networks: Semi-automated methods

- Grid search
  - Finds optimal hyperparemeters (within grid), but very computationally expensive
- Random search (of a subset of the grid)
  - Often does pretty well, and much faster, but can miss optimal configuration
- Bayesian Optimization
  - Uses probabilistic models to predict which hyperparameters will improve performance, focuses on tuning those – best when each model is costly to train
  - **Example**: If one learning rate produces poor performance, Bayesian optimization will avoid similar values and explore more promising areas
  - **Pro**: More efficient than grid and random search, converges faster to optimal values
  - **Cons**: Requires additional computation to model probability distributions

Tuning networks: Semi-automated methods

- There are other (less common) methods too!

Method	Best for	Pros	Cons
Grid Search	Small hyperparameter spaces	Exhaustive search, easy to implement	Computationally expensive
Random Search	Large hyperparameter spaces	Faster than grid search	Might miss optimal values
Bayesian Optimization	Efficient search for good results	Uses probabilistic modeling to find the best values efficiently	Computational overhead
Genetic Algorithms	Complex hyperparameter spaces	Can explore non-obvious configurations	Slow, needs many iterations
Hyperband	Large-scale deep learning tuning	Stops poor configurations early, saving time	Can discard good models too soon
Population-Based Training	Adaptive tuning for deep learning	Adjusts hyperparameters dynamically	Requires parallel computing
RL-Based Tuning	Complex, high-dimensional spaces	Learns best hyperparameters automatically	Requires expertise in reinforcement learning
Neural Architecture Search	Optimizing entire neural network designs	Finds optimal architectures	Requires massive computing power

Tuning Networks: Modern tools

- Bayesian optimization tools
  - Scikit-Optimize, Ax (Meta), Spearmint, GPyOpt, BayesianOptimization
- Hyperparameter optimization tools (more user-friendly)
  - Optuna, Hyperopt, Ray Tune, SKopt, SigOpt, Keras Tuner, GridSearchCV
- AutoML
  - Auto-sklearn, TPOT, FLAML, AutoKeras, Google AutoML, H20 AutoML
- Experiment monitoring and visualization
  - Weights and biases, TensorBoard, MLflow, Neptune.ai, Comet, Sacred

Key Takeaways

- Perceptrons cannot represent nonlinear decision boundaries
- Multilayer networks can represent complex functions
- Learning requires differentiable nonlinear activations
- Backpropagation computes gradients using the chain rule
- Training networks is subtle
- Final networks can be very difficult to interpret